

CSAI2017P — Applied Machine Learning

Test-0 — Diagnostic • Solutions

Sat 10 August 2026 • 30 minutes • This test carried no marks

These are the answers to the diagnostic you sat in week 2, with the reasoning behind each one.

The test carried no marks and none were awarded, so there is nothing here to check a score against. What it is for is the other thing: you can now see how far your answer was from a complete one, and which of the six blocks you want to shore up before the material that depends on it arrives.

Several of these questions have more than one good route to the answer, and a route different from the one printed here is not automatically a worse one. Where a second route is worth seeing, it is given. If you reached a different answer and can show your reasoning, bring your script.

Block A — Warm-up

5 items, single answer

A1. A fair coin is tossed twice. The probability of exactly one head.

(b) $1/2$. The four outcomes HH, HT, TH, TT are equally likely, and two of them — HT and TH — have exactly one head, so the probability is $2/4$. The common wrong answer is $1/4$, which counts HT but forgets that TH is a different outcome with the same number of heads.

A2. `len({'a': 1, 'b': 2})` evaluates to:

(b) **2**. The braces here build a *dictionary* with two key–value pairs, and `len` of a dictionary counts its keys, not its keys and values separately. Hence 2, not 4.

A3. A is 3×4 and v is a column vector. For Av to be defined, v must be:

(b) 4×1 . In a product AB the inner dimensions must agree: $(3 \times 4)(4 \times 1)$ works and gives a 3×1 result. The rule to carry forward is that the number of *columns* of A must equal the number of *rows* of v — which is exactly the check you will be doing on every design matrix later in the course.

A4. For 2, 3, 4, 5, 96, which is larger, the mean or the median?

(a) **the mean**. The mean is $(2 + 3 + 4 + 5 + 96)/5 = 110/5 = 22$; the median is the middle value of the sorted list, 4. One large value drags the mean a long way and leaves the median where it was. This is the same fact that decides, later in the course, why squared error is fitted by the mean and absolute error by the median.

A5. A *training set* is:

(a) **data used to fit the model**. The final performance figure must come from data the model has not been fitted on, which is what a test set is for; and a training set is not defined by whether it carries labels.

Block B — Working

3 items

B1. Machine 1 makes 60% of items, 2% defective; machine 2 makes the rest, 5% defective. An item is found defective. How likely is it that machine 1 made it?

0.375, **that is** $3/8$. The quickest honest route is to stop using percentages and imagine a concrete batch — say 1000 items.

Machine 1 makes 600 of them, and 2% of $600 = 12$ are defective.

Machine 2 makes 400 of them, and 5% of $400 = 20$ are defective.

Defective items in total: $12 + 20 = 32$, of which 12 came from machine 1.

So the answer is $12/32 = 0.375$. In Bayes' notation the same calculation is

$$P(M_1 | D) = \frac{P(D | M_1) P(M_1)}{P(D | M_1) P(M_1) + P(D | M_2) P(M_2)} = \frac{0.02 \times 0.6}{0.02 \times 0.6 + 0.05 \times 0.4} = \frac{0.012}{0.032} = 0.375.$$

The sanity check is the part worth having. Machine 1 makes *more* of the items but is less than half as likely to spoil one, so on being told an item is defective you should expect the answer to come out below 50% — and it does. An answer of 0.6 means the question of which machine made it has been confused with the question of which machine spoiled it.

B2. Find the c minimising $S(c) = \sum_{i=1}^n (y_i - c)^2$, and say how you know it is a minimum.

$c = \bar{y}$, **the mean of the y_i** . Differentiating,

$$S'(c) = \sum_{i=1}^n 2(y_i - c)(-1) = -2 \sum_{i=1}^n (y_i - c) = 0 \implies \sum_{i=1}^n y_i = nc \implies c = \frac{1}{n} \sum_{i=1}^n y_i.$$

It is a minimum because $S''(c) = 2n > 0$ for every c — the second-order condition, which the question asked for explicitly and which a majority of scripts left out.

A route with no calculus in it. Add and subtract $\bar{y}$ inside the bracket and expand; the cross term vanishes because $\sum (y_i - \bar{y}) = 0$, leaving

$$S(c) = \sum_i (y_i - \bar{y})^2 + n(c - \bar{y})^2.$$

The first term does not involve c at all and the second is n times a square, so $S(c)$ is an upward parabola in c whose least possible value is reached exactly when $c = \bar{y}$. This form also shows why the minimum is unique, and it is worth remembering — the same decomposition returns as the bias–variance identity later in the course.

B3. For the given `summarise`, what does `summarise([2, 4, 6])` return, and what does `summarise([])` do?

`summarise([2, 4, 6])` returns 4.0. Tracing the loop: `total` runs $0 \rightarrow 2 \rightarrow 6 \rightarrow 12$, and the function returns $12/3$. Note the answer is the *float* 4.0 and not the integer 4: in Python 3 the `/` operator always performs true division, and `//` is the one that would give 4.

`summarise([])` raises `ZeroDivisionError: division by zero`. The loop body never executes, so `total` is still 0; then `len(values)` is 0 and the final line evaluates $0 / 0$. It does not return 0, and it does not return `None` — it raises, and the function has no guard against it.

The habit being asked for is tracing a short function by hand on a specific input rather than reading it for what it looks like it does. The empty list is the standard edge case for anything that divides by a count, and it is worth reaching for automatically.

Block C — Explain it

1 item

C1. In at most 120 words, explain overfitting to a first-year student, with one concrete example.

There is no single correct wording here, and the form of the explanation was left to you deliberately — an analogy, a sketch, a small table of training and test error, or a definition are all legitimate. A complete answer contains three things: what the model is doing, why that is a problem, and an instance you can point at. One version:

A model overfits when it learns the accidents of the data it was trained on — the noise, the particular rows it happened to see — instead of the pattern that would hold on new data. You notice it by the gap: error on the training data keeps falling while error on data the model has never seen starts to rise. Fitting a high-degree polynomial to a handful of noisy points is the standard picture; the curve passes exactly through every point and is useless a little way beyond them. The everyday version is a student who has memorised past papers rather than the subject, and meets a question phrased differently.

The give-away that an answer has not landed is defining overfitting as “the model is too accurate” or “the model is too complex”. Complexity is a common *cause*; the thing itself is the gap between performance on seen and unseen data, which is why the definition has to mention both.

Block D — Debug

1 item

D1. One fault in `evaluate`. State what is wrong, why it matters, and how you would confirm it.

(i) What is wrong. The scaler is fitted on the *whole* of `X` before the split:

```
X_scaled = scaler.fit_transform(X)          # <-- sees every row, test rows included
X_train, X_test, ... = train_test_split(X_scaled, y, ...)
```

`fit_transform` computes the mean and standard deviation of each column from all the rows and then rescales with them. The rows that become the test set have therefore already contributed to the numbers used to transform the training set. That is leakage: information from the test set has reached the model before it was evaluated.

(ii) Why it matters. The figure the function returns is no longer an estimate of performance on unseen data, and it is optimistic — the test rows have been mapped using statistics they helped to define. The effect is worst exactly where you can least afford it: small datasets, and columns with outliers or heavy tails, where a handful of test rows can move a column’s mean or spread appreciably. It matters more here than it would for many models, because k -nearest-neighbours decides everything by distance, so the scaling *is* the model’s view of the data.

(iii) How to confirm it. Split first, then fit the scaler on the training rows only and merely `transform` the test rows, and compare the two scores:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)
scaler = StandardScaler().fit(X_train)        # fitted on training rows only
model = KNeighborsClassifier(n_neighbors=5).fit(scaler.transform(X_train), y_train)
model.score(scaler.transform(X_test), y_test)
```

If the score drops, the leak was inflating it. The tidier form of the same fix is a `Pipeline([('scaler', StandardScaler()), ('knn', KNeighborsClassifier())])` passed to `cross_val_score`, which re-fits the scaler inside every fold and makes the mistake structurally hard to repeat.

Two things that are *not* the fault, both named in a number of scripts: `test_size=0.25` and `n_neighbors=5` are choices, not errors, and `random_state=0` is what makes the result reproducible rather than what breaks it.

Block E — Getting started

1 item

E1. A shop wants to predict whether a customer will return next month. Describe how you would start.

The question is under-specified on purpose, and the efficient first move is to say so. Naming an algorithm straight away is not wrong, but it commits to a shape of solution before the problem has one. The things that have to be settled before any model can be fitted:

- **What “return” means.** A purchase? A visit? Within a calendar month, or 30 days from a fixed date? This is the target column, and nothing can be built until it is a rule you could apply to a row by hand.
- **What data exists**, and how far back — transactions with dates and a customer identifier at minimum. Without a way to link a purchase to a person there is no problem to solve.
- **What the base rate is.** If 90% of customers return anyway, a model that predicts “yes” for everybody is 90% accurate and worthless, and you need to know that before you report an accuracy.
- **What a wrong prediction costs** in each direction, and **what the prediction is for** — a voucher sent to someone who was coming anyway is cheap; missing a customer who was about to leave may not be.

Then the shape of a first attempt: pick a cut-off date, define the label from what happened in the month *after* it, and build features only from data *before* it — recency, frequency and spend are the obvious three. Because the data are ordered in time, evaluate on a later period rather than on a random split, or you are predicting the past from the future. Start with a baseline dull enough to be beaten — “predict that whoever came last month comes again” — and treat any model that cannot beat it as not yet working.

Block F — Estimation

1 item

F1. Roughly how many numbers are in a 1000×1000 greyscale image, and how much memory would 500 of them need?

10^6 numbers per image; 5×10^8 numbers in total; about 500 MB at one byte per pixel. Greyscale means one number per pixel, so the count is $1000 \times 1000 = 10^6$, and $500 \times 10^6 = 5 \times 10^8$ numbers altogether.

Turning that into memory needs an assumption, and **stating it was the point of the question**: an 8-bit greyscale pixel is one byte, so 5×10^8 bytes $\approx$ 500 MB. Load the same data as `float64` — which is what happens by default if you convert to a NumPy float array without thinking — and each number takes 8 bytes instead, giving 4×10^9 bytes $\approx$ 4 GB.

That factor of eight is the practical content of the question. 500 MB fits in memory on any laptop; 4 GB may not, and the fix is either to keep the data as `uint8` until the moment of use or to process it in batches. An answer of 5×10^8 with no unit and no assumption is only half of one, and an answer in the region of megabytes-per-image or terabytes-in-total has lost a factor of a thousand somewhere — which is what checking the magnitude at the end is for.

Test-0 carried no marks and none were recorded. Questions on any of this are welcome in the next class or in office hours.